1. Descripción Breve del Sistema y del Módulo MVP

- Sistema General (SHOP): Plataforma de comercio electrónico diseñada para permitir a clientes registrarse, autenticarse, consultar un catálogo con paginación y filtros, realizar compras y a administradores gestionar productos, subir archivos y utilizar mensajería en tiempo real mediante WebSockets. El backend está construido en NestJS, TypeORM y PostgreSQL, mientras que el frontend es una SPA desarrollada con React, Vite, TypeScript y Zustand.
- Módulo MVP (Auth): Encargado de la gestión de identidad, control de acceso y vigencia de sesiones de usuario. Comprende las rutas de registro (POST /api/auth/register), inicio de sesión (POST /api/auth/login) y verificación/renovación del estado de sesión (GET /api/auth/check-status). La autenticación se basa en JSON Web Tokens (JWT) con Passport (vigencia de 2 horas) y la autorización se gestiona por roles (admin, super-user, user).

2. Vista Lógica Final del Sistema

La arquitectura lógica del módulo de autenticación está estructurada en capas independientes con responsabilidades desacopladas:

- Capa de Presentación:
  - Frontend (React + Zustand): Manejo del formulario de login/registro (LoginPage, RegisterPage), estado global de sesión mediante useAuthStore y protección de rutas con ProtectedRoutes.
  - Backend (AuthController y DTOs): Exposición de la API REST. Recibe datos validados por DTOs de entrada (CreateUserDto, LoginUserDto) y transforma la respuesta serializada mediante DTOs de salida (AuthResponseDto, UserResponseDto).
- Capa de Control de Acceso:
  - Decoradores y Guards (@Auth, @RoleProtected, @GetUser, UserRoleGuard): Interceptan las solicitudes HTTP para validar los roles requeridos según la metadata y extraer el usuario autenticado.
  - Estrategia Passport (JwtStrategy): Valida la firma del JWT, comprueba su expiración y recupera el estado activo del usuario en la base de datos.
- Capa de Aplicación:
  - AuthService: Ejecuta la lógica central de negocio (cifrado con bcrypt, validación de credenciales, manejo de excepciones de duplicidad 23505 y firma de tokens JWT a través de JwtService).
- Capa de Persistencia:
  - UserEntity + Repository<User>: Representación persistente del usuario en PostgreSQL mediante TypeORM (campos: id, email, password, fullName, isActive, roles).

3. Diagramas C4 del Módulo MVP

Nivel 1: Diagrama de Contexto

- Usuarios: Cliente / Administrador.

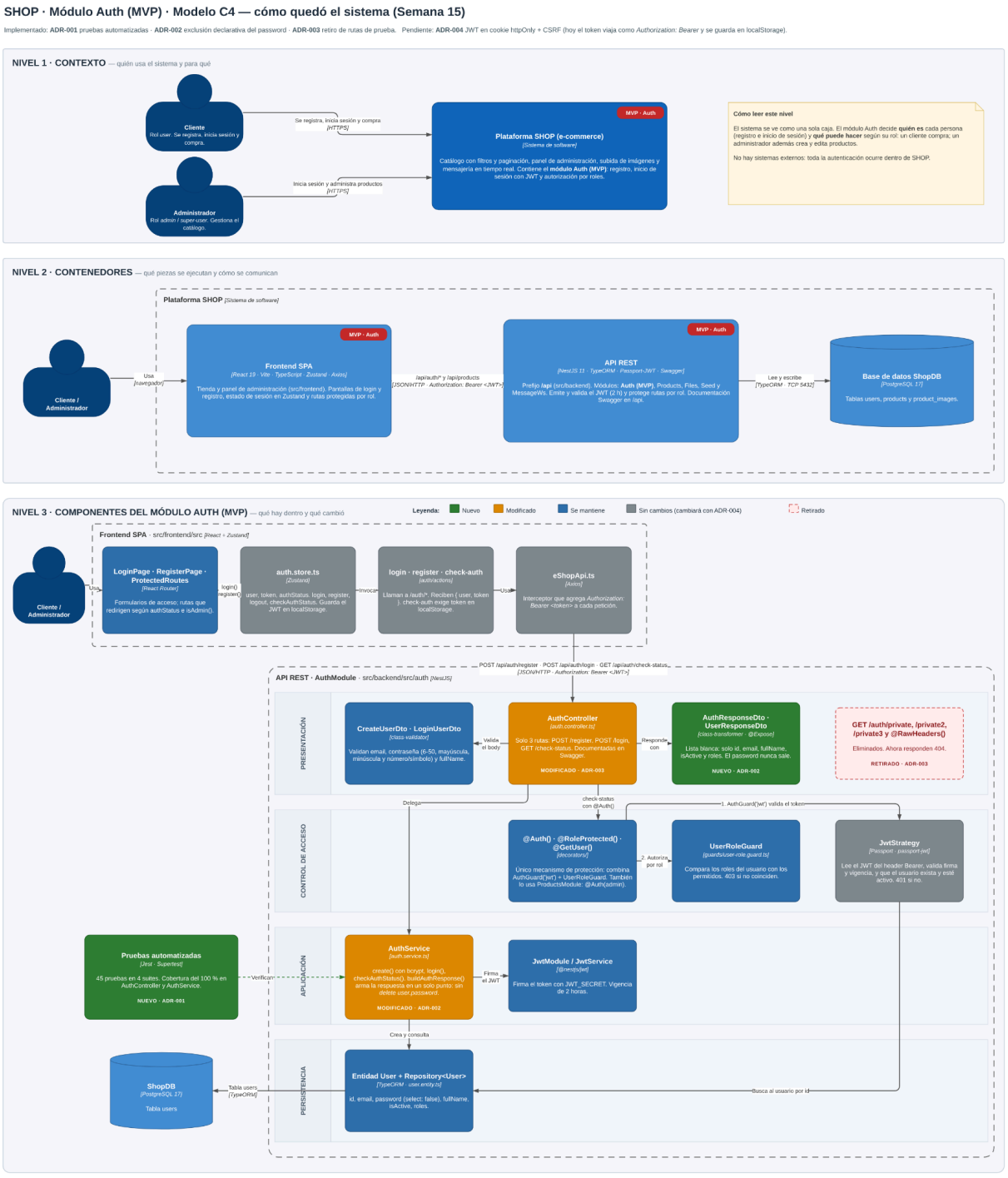
- Sistema: Plataforma SHOP (E-commerce) — El módulo Auth decide quién es cada persona y qué acciones puede realizar según su rol asignado (comprar, gestionar productos).
- Relaciones: Los clientes y administradores interactúan con la interfaz web para autenticarse, consultar catálogo y administrar la tienda.

Nivel 2: Diagrama de Contenedores

- Frontend SPA (React + Vite + Zustand): Aplicación cliente ejecutada en el navegador que gestiona las vistas de autenticación, el estado de sesión y las rutas protegidas. Comunica mediante JSON/HTTP enviando tokens Bearer.
- API REST (NestJS + TypeORM): Servidor de aplicaciones que procesa las peticiones de autenticación, ejecuta la lógica de roles, aplica validaciones y emite JWTs de 2 horas.
- Base de Datos ShopDB (PostgreSQL 17): Almacena las tablas de usuarios (users), productos y relaciones.

Nivel 3: Diagrama de Componentes (Módulo Auth)

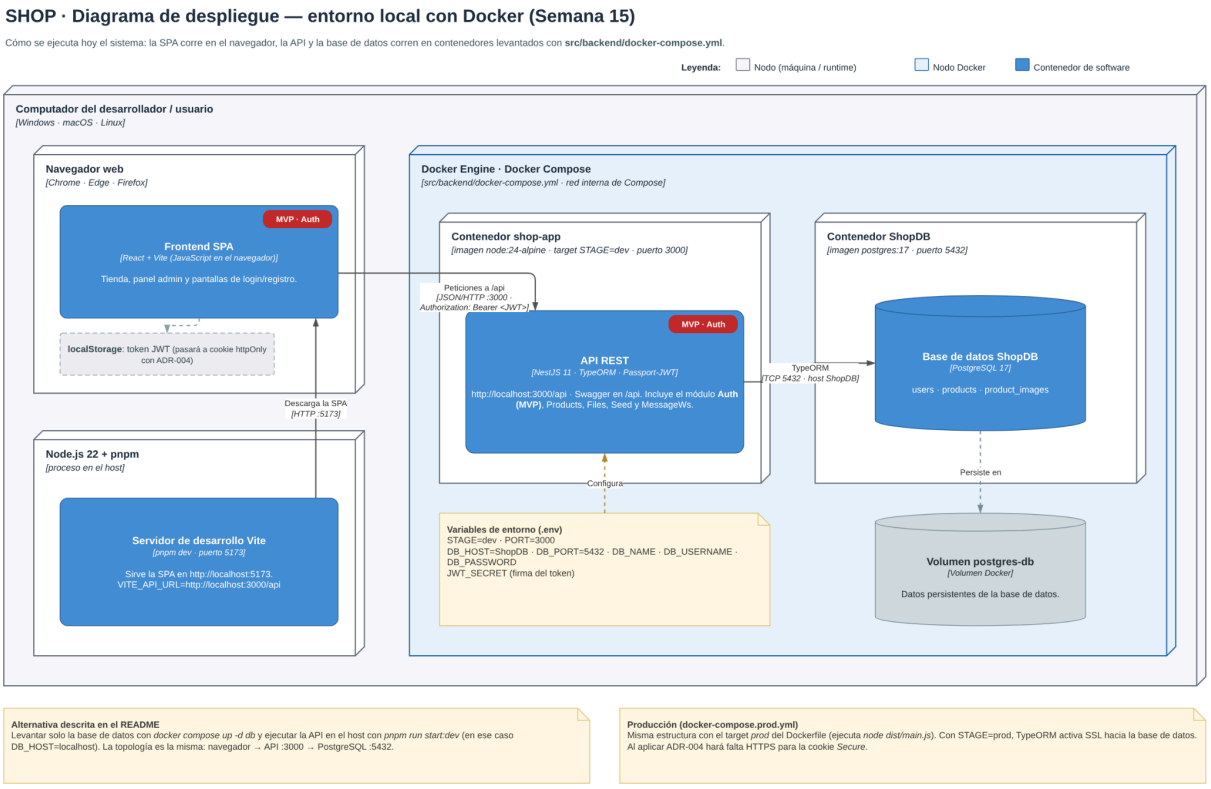
- Componentes Frontend: LoginPage/RegisterPage → auth.store.ts → login/register/check-auth actions → eShopApi.ts.
- Componentes Backend (Presentación): AuthController consume CreateUserDto y LoginUserDto, y responde con AuthResponseDto / UserResponseDto.
- Componentes Backend (Control de Acceso): @Auth unifica @RoleProtected + AuthGuard('jwt') + UserRoleGuard. JwtStrategy valida la firma del token.
- Componentes Backend (Aplicación y Persistencia): AuthService interactúa con JwtService y Repository<User> (UserEntity) conectado a ShopDB.
- Infraestructura de Pruebas: Suite de pruebas en Jest (auth.service.spec.ts, jwt.strategy.spec.ts, user-role.guard.spec.ts, auth.integration.spec.ts).



4. Diagrama de Despliegue (Entorno Local / Docker)

- Navegador Web (Host del Desarrollador): Ejecuta la SPA servida en http://localhost:5173 por el servidor de desarrollo Vite (o construida estáticamente).
- Red Interna de Docker (docker-compose.yml):
    - Contenedor shop-app: Ejecuta la API REST en Node.js 24 Alpine (STAGE=dev, Puerto 3000), exponiendo Swagger en /api.
    - Contenedor ShopDB: PostgreSQL 17 exponiendo el puerto 5432 hacia la red de Compose (o mapped al host).
    - Volumen Docker postgres-db: Mantiene la persistencia de datos.

- Variables de Entorno (.env): STAGE, PORT, DB_HOST, DB_PORT, DB_NAME, DB_USERNAME, DB_PASSWORD, JWT_SECRET.



5. Refactorización Realizada

5.1. Problemas de la Versión Inicial

1. Rutas de prueba expuestas: Exposición en entorno productivo de los endpoints de prueba GET /auth/private, /private2 y /private3, los cuales imprimían información interna y encabezados crudos (@RawHeaders) en consola.
2. Exclusión manual e insegura de contraseñas: El hash del password se eliminaba manualmente de la respuesta HTTP mediante delete user.password en el servicio.
3. Almacenamiento del JWT en localStorage: Riesgo de exposición frente a ataques de scripts entre sitios (XSS) al almacenar la credencial en el almacenamiento local del navegador.
4. Ausencia de pruebas automatizadas: El módulo no contaba con suites de pruebas unitarias o de integración (0 tests found).

5.2. Cambios Aplicados en el Diseño y el Código

- Capas y DTOs de Salida (ADR-002 / ADR-001): Se implementaron UserResponseDto y AuthResponseDto utilizando @Expose() de class-transformer. Se eliminó delete user.password en favor de la serialización declarativa con lista blanca.
- Limpieza de Endpoints (ADR-003): Se eliminaron definitivamente las rutas /private* del controlador AuthController y el decorador @RawHeaders, reduciendo el área de ataque y limpiando la especificación de Swagger.

- Estrategia de Pruebas Automatizadas (ADR-001): Se construyó una suite de 45 pruebas distribuidas en 4 suites con Jest:
    - Unitarias: auth.service.spec.ts (bcrypt, JWT, manejo de errores 400/401/500), jwt.strategy.spec.ts (validación de usuarios activos e inactivos), user-role.guard.spec.ts (matriz de permisos por rol).
    - Integración HTTP: auth.integration.spec.ts (flujos completos de registro, login, check-status y verificación del retorno 404 en rutas eliminadas). Cobertura del 100% en AuthController y AuthService (89.67% de sentencias en el módulo Auth).
- Alineación de decisiones pendientes (ADR-004): Aceptación en diseño de la migración de JWT a cookies httpOnly + Secure + SameSite con protección CSRF en backend y frontend.

5.3. Mejora de Atributos de Calidad Priorizados

Atributo de Calidad	Escenario / Justificación de la Mejora
Seguridad	Escenario 1: Eliminación de la superficie expuesta por endpoints /private*. Exclusión garantizada de hashes de contraseña en el payload de respuesta. Diseño alineado a cookies httpOnly para mitigar vectores XSS.
Mantenibilidad	Escenario 2: Desacoplamiento de la transformación de datos. El paso a DTOs explícitos (@Expose) evita modificaciones propensas a error manual (delete user.password) en la lógica de negocio (AuthService).
Testabilidad	Escenario 3: Cobertura automatizada del 100% en controladores y servicios de autenticación mediante Jest, permitiendo refactorizaciones seguras en el pipeline sin depender de ejecuciones de base de datos para la suite Auth.